Adam v1: A Repo-Grounded Whitepaper for a Local-First Graph-Conditioned Runtime

Codex for the Adam repository

March 20, 2026

Abstract

This paper revises the Adam v1 whitepaper against the repository that currently exists rather than against inherited conceptual prose. The resulting artifact is Adam-first in naming, repo-grounded in evidence, and explicit about discontinuities between historical claims and current implementation. The core result is a direct local-first runtime in which persistent identity is externalized into graph state, explicit feedback, bounded retrieval assembly, and attributable observability rather than into model-weight training. In the current repository, memes are first-class persistent graph units, memodes are derived second-order structures with an admissibility floor, regard is durable selection pressure distinct from prompt-time salience, the membrane is a post-generation control surface with persistent trace effects, the terminal user interface is the primary runtime instrument, and the browser observatory is a secondary measurement and mutation workbench with an explicit authority boundary. The paper compiles those claims only where the repository yields code, tests, runtime traces, or export artifacts strong enough to sustain implementation-strength language. It also treats the repo's own document field as data: the designated baseline draft still carries a governor-first recursive architecture and a looser meme/memode substrate than the current code and norms support. The whitepaper therefore preserves continuity where the evidence survives, downgrades older language where proof has narrowed, and tombstones obsolete architectural claims instead of silently smoothing them into the present tense.

1 Introduction

Adam is the current runtime identity described by this paper. The repository still contains legacy `EDEN` strings in filenames, module paths, and README headers, but the operative public name for the current runtime is Adam, and the repo evidence supports treating those older strings as implementation-history anchors rather than as the best name for the present system.[1] The distinction matters because earlier prose allowed the shell/runtime boundary to blur into a looser story about an Eden substrate containing Adam as an instance. The current repo is sharper than that. The strongest claim that survives current evidence is not that a general Eden architecture has already been realized in full public form, but that Adam v1 currently operates as a local-first graph-conditioned runtime with a bounded set of measurement and observability instruments.

The methodological stance of this paper is therefore intentionally strict. Prose is treated as executable specification only where it cashes out in code, tests, logs, exports, or other reproducible evidence. The paper does not inherit implementation strength from prior drafts, from the current prompt family, or from memetic rhetoric. When the repo has stronger evidence than earlier prose, the prose is revised to match the repo. When the repo has weaker evidence than earlier prose, the prose is downgraded rather than the repository being rhetorically upgraded. This is not merely a stylistic choice. Adam is a system whose public intelligibility depends on boundary maintenance. If the paper collapses derived structures into first-class ones, browser-local state into evidence, or historical aspiration into present-tense fact, it stops being a useful second implementation surface and becomes marketing copy masquerading as technical writing.

Within that constraint, the repository does support a coherent and scientifically legible architecture. Adam is local-first in the concrete sense that the active runtime is tied to a repo-managed MLX model and a local SQLite-backed persistence layer rather than to remote inference or cloud-hosted memory services.[2] Persistent identity is externalized rather than learned into weights: the graph store, explicit feedback events, active-set assembly, membrane traces, and observatory measurements do the work that other systems often ascribe to opaque latent

[1] Naming continuity and legacy-shell drift are visible in `README.md:L3-L7`, which still uses "EDEN / ADAM v1.2", while the current pipeline artifacts and this run are Adam-first. The baseline draft under revision is `assets/white_paper_pipeline/white_paper_drafts/eden_whitepaper_v14.pdf`, but it is used as a drift surface rather than as implementation proof.

[2] The local MLX commitment is explicit in `docs/PROJECT_CHARTER.md:L16-L19` and operationalized in `README.md:L32-L38, L63-L65`. The current inference contract lives in `eden/inference.py:L10-L18,L295-L340`.

state or fine-tuned model adaptation.[3] The base model is therefore treated as an inference surface rather than as the substrate in which Adam's identity is stored. The persistent substrate is the graph plus its event surfaces.

That distinction leads to a second major result: the causal story of Adam v1 is direct rather than governor-mediated. The current v1 loop is "input → retrieve/assemble → Adam response → membrane → feedback → graph update."[4] The paper will show that this matters operationally, because the currently designated baseline manuscript still describes a governor-first architecture in which every turn begins with a planning layer. The current repo does not implement that architecture. What it does implement is a bounded retrieval and prompt-compilation process, a visible membrane, explicit operator feedback, regard-driven persistence pressure, and an observatory that can inspect and, under the live server contract, preview and commit topology changes with provenance.

The third result is about instrumentation rather than architecture in the narrow sense. Adam's browser observatory is real, but it is not the main dialogue loop. The terminal user interface remains the primary runtime instrument; the browser surface is a secondary measurement workbench that exposes graph, geometry, basin, measurement, and runtime-trace views, while preserving a strong boundary between browser-local view state and authoritative persistent mutation.[5] This matters because the public temptation is to overstate browser capability whenever a repo contains a sophisticated React workbench. The current code is more disciplined than that temptation: browser-local layout presets, preview styling, and table state are explicitly non-evidentiary, while mutation authority routes through live preview/commit/revert endpoints and persists measurement events.[6]

The remainder of the paper proceeds in five moves. Section 2 stabilizes Adam's canonical terms and non-collapse rules. Section 3 compiles the current direct-loop architecture, persistence substrate, and selection mechanics. Section 4 explains the TUI/observatory split and the browser authority boundary. Section 5 reports the strongest observed operational evidence available in the current repo, including tests refreshed in this run and runtime/export artifacts already present in the project. Section 6 treats the repository's own document field as archaeological data and shows where prior language outruns current proof. Section 7 closes with limitations and closure work, phrased as future or deferred work where the repo still lacks implementation-strength evidence.

2 Canonical Terms and Non-Collapse Rules

The current repository does not permit a loose or purely rhetorical use of its core terms. A *meme* is the first-class persistent unit at the graph layer. In the current compatibility store, both behavior-domain and knowledge-domain rows still inhabit the `memes` table, but the ontology projects knowledge-domain rows outward as *information* and behavior-domain rows outward as *behavior memes*.[7] That distinction is not cosmetic. It defines what counts as a behavior-bearing unit, what is eligible for memode materialization, and what kinds of edges are admissible as support rather than merely informational relations. A positive example is a feedback-derived behavior meme whose graph relations participate in qualifying support edges; a near miss is a knowledge row about an author or citation target. Both can persist, but they do not play the same role in the identity story.

A *memode* is stricter still. The current ontology defines a memode as a derived second-order structure built from at least two behavior-domain memes plus a connected qualifying support subgraph.[8] The positive example is an audited behavior-domain assembly whose member memes are connected through support edges and pass the admissibility checks exported in the memode audit surface. A near miss is a semantic cluster label or community summary: the repo explicitly says that clusters are not memodes, that knowledge relations do not silently become memode support, and that wake-up or observability summaries are not automatically promoted into durable second-order behavior objects.[9] The failure mode this rule prevents is conceptual inflation. If every graph neighborhood, cluster, or summary label could be called a memode, the term would stop carrying any operational boundary. The current repo protects against that collapse.

Regard is likewise a term of art rather than a synonym for importance. In Adam v1, regard names durable selection pressure over persistent graph objects. It is not identical to prompt-time salience, active-set membership, or one-

[3] The repo's explicit scope boundary is in `docs/PROJECT_CHARTER.md:L16-L19`; the status surface marks training and hidden planner claims as absent or deferred in `docs/IMPLEMENTATION_TRUTH_TABLE.md:L90-L92`.

[4] The turn-order contract is explicit in `docs/TURN_LOOP_AND_MEMBRANE.md:L7-L12`. The profile contract also says that no hidden governor is introduced in `docs/INFERENCE_PROFILES.md:L22-L23,L134-L135`.

[5] The TUI priority is stated in `docs/PROJECT_CHARTER.md:L18-L19` and elaborated in `README.md:L63-L65` and `docs/TUI_SPEC.md`. The browser contract and its authority boundary are specified in `docs/OBSERVATORY_SPEC.md:L187-L216,L320-L343` and `docs/OBSERVATORY_INTERACTION_SPEC.md:L67-L96`.

[6] See `docs/MEASUREMENT_EVENT_MODEL.md:L51-L57` for the persistence boundary and `web/observatory/src/App.tsx:L853-L871,L2258-L2270,L2365-L2379` for the current browser-side authority checks and static-mode disclosures.

[7] The current ontology draws that distinction in `docs/CANONICAL_ONTOLOGY.md:L10-L18`. The graph schema keeps the compatibility-table detail explicit in `docs/GRAPH_SCHEMA.md:L82-L85`.

[8] The core definition is in `docs/CANONICAL_ONTOLOGY.md:L20-L24,L51-L57`. The audit distinction between support relations and informational relations is visible in the current export surface summarized in Figure 3.

[9] `docs/CANONICAL_ONTOLOGY.md:L24,L52-L57`.

off relevance to the current user turn. A meme can be highly salient in a given turn while having low durable regard, and a high-regard meme can remain outside the current prompt if the retrieval aperture, mode, or current query does not select it.[10] The positive example is a graph item strengthened by explicit feedback and later retrieval pressure. The near miss is an item that appears once in a prompt because the user just said it. This distinction matters because older memetic language can tempt writers to treat every activation-like phenomenon as selection. Adam's current implementation is more concrete than that: selection pressure persists in graph state and evidence-bearing event surfaces, while the active set is a bounded prompt-assembly artifact.

The *active set* is the bounded retrieval and prompt-compilation slice that the runtime assembles for a turn. It is not a hidden governor, not a latent planning layer, and not the full identity substrate. The active set is the runtime's current working slice of persistent graph material under budget and profile constraints.[11] The positive example is a preview surface or log event that reports an active-set size before generation. The near miss is a claim that the active set *is* Adam. The repo never licenses that stronger sentence, because Adam's continuity is distributed across the graph, feedback, regard, traces, and repeated retrieval rather than collapsed into any single ephemeral slice.

The *membrane* is the operator-visible post-generation control surface. In the current implementation it sanitizes the raw model output, enforces bounded operator-facing response form, and records visible membrane events into persistent storage.[12] A positive example is the e2e test that proves the final operator-facing response excludes scaffold headings such as "Answer:", "Basis:", and "Next Step:" after sanitization.[13] The near miss is any claim that the membrane is a hidden planner or a latent cognition layer. The repo is explicit that it is a post-generation control and persistence surface, not a pre-generation governor.

Feedback events, *trace events*, *membrane events*, and *measurement events* form the event family through which Adam makes persistence inspectable. Feedback events carry explicit operator verdicts and, for accept/reject/edit, explanations or corrected text where required. Trace and membrane events bind runtime actions to observable surfaces. Measurement events bridge observation and graph mutation in the observatory contract.[14] The positive example is a committed observatory edge-add event followed by a distinct revert event, both persisted in the exported ledger. The near miss is browser-local layout state or preview styling; the repo states plainly that those remain browser-local and are never measurement events.[15]

The *observatory* is the browser-side measurement and inspection instrument. It is not the main dialogue surface, not a general graph IDE, and not evidence by default simply because it renders a graph attractively. The current browser shell exposes graph, data-laboratory, and preview workspaces; it can display live or static payloads; and when live endpoints are present it can preview, commit, and revert attributable changes.[16] But the observatory is bounded in two important ways. First, the repo keeps Python authoritative for exports, clustering, basin projection, and measurement provenance. Second, the browser exposes both live and static modes and says when mutation chrome is visible but disabled.[17] The paper therefore uses the phrase *browser contract gap* when a server-side capability exists but current browser exposure is partial or mode-limited. That phrase is not rhetoric; it is a guardrail against overstating the browser surface.

Finally, the current repo uses the term *hum* in a deliberately narrow way. The hum is a bounded continuity artifact derived from persistent active-set, feedback, and membrane-related surfaces. It is not an input to generation, not a hidden inner voice, not a planner residue, and not identical to the active set or the whole graph.[18] The positive example is the current transcript and hum export family where a session's continuity artifact is present, timestamped, and path-addressable. The near miss is a claim that the hum is a second mind operating alongside Adam. The current repository does not support that stronger sentence.

3 Direct-Loop Architecture and Persistent Selection

With those definitions in place, Adam's current architecture can be stated plainly. A session begins with operator input through the TUI. The runtime retrieves and assembles a bounded prompt context, produces a model response on the local backend, applies the membrane to that

[10] The repo's distinction between durable selection and prompt-time assembly is stabilized in `docs/REGARD_MECHANISM.md` and in the active-set assembly contract of `docs/TURN_LOOP_AND_MEMBRANE.md:L57-L71`.

[11] The budgeted assembly order is defined in `docs/TURN_LOOP_AND_MEMBRANE.md:L7-L12,L57-L71`. The mode/profile boundaries are implemented in `eden/inference.py:L10-L18,L177-L186,L295-L314`.

[12] What the membrane does and does not do is specified in `docs/TURN_LOOP_AND_MEMBRANE.md:L37-L46`. The sanitization implementation is anchored at `eden/runtime.py:L2069-L2152`.

[13] `tests/test_runtime_e2e.py:L16-L33`.

[14] The measurement-event bridge is defined in `docs/MEASUREMENT_EVENT_MODEL.md:L3-L17,L46-L57`. Feedback protocol and explanation requirements are specified in `docs/EXPERIMENT_PROTOCOLS.md`.

[15] `docs/MEASUREMENT_EVENT_MODEL.md:L51-L57`; `docs/OBSERVATORY_INTERACTION_SPEC.md:L138-L147`.

[16] See `docs/OBSERVATORY_SPEC.md:L3-L18,L187-L216,L320-L343`.

[17] `docs/KNOWN_LIMITATIONS.md:L21-L25,L48-L54`; `web/observatory/src/App.tsx:L853-L871,L2258-L2270,L3868-L3870`.

[18] The narrow hum contract is explicit in `docs/KNOWN_LIMITATIONS.md:L44-L45` and in the hum tests at `tests/test_hum_runtime.py:L35-L63,L92-L121`.

response, persists turn data and membrane-related surfaces, accepts explicit feedback, and then updates graph channels such that later retrieval pressure can change.[19] This is the direct v1 loop. It is the architectural spine the current code actually supports.

The first architectural fact to stress is what is absent. Adam v1 has no governor in the current repo, no hidden planner, no weight training or fine-tuning path, and no LoRA-backed persistence strategy.[20] This absence is not a gap accidentally left undocumented; it is an active constitutional boundary. The repo even records that `adam_auto` on the MLX path currently falls back to `runtime_auto` and that there is no hidden Adam prepass in the present build.[21] That fallback becomes operationally visible in the runtime log, where a generation request recorded as `requested_mode = adam_auto` is logged with `effective_mode = runtime_auto` on the MLX backend.[22]

That fallback is instructive because it shows how Adam's current honesty discipline works. The repo does not bury the fallback in an internal heuristic or pretend a more elaborate chooser exists. Instead, it records the requested mode, the effective mode, and the resulting profile name as visible runtime fields.[23] This is a recurring design pattern throughout Adam v1. When a richer future mechanism is not yet implemented, the current build prefers a bounded fallback plus explicit persistence over a theatrical simulation of depth. The same pattern recurs in geometry, where large graphs can fall back to sparse-safe approximations, and in observatory static mode, where mutation chrome can stay visible yet disabled instead of being mislabeled as available authority.[24]

The second architectural fact is that persistent identity is externalized. Adam remains the same runtime over time not because a model has been fine-tuned into a personalized basin, but because turns, feedback, documents, edges, membrane events, and measurements accumulate in persistent graph state, and later retrieval assembly is pressured by that state.[25] The e2e runtime test makes that causal story concrete: a single chat turn increases turn and meme counts, produces a bounded active set, yields a membrane-cleaned response, accepts edit feedback with corrected text, and then allows subsequent retrieval to find the feedback-derived material.[26] The persistent substrate is therefore not an abstract graph promised by documentation; it is a store whose cardinalities, event families, and export surfaces can be counted directly.[27]

This externalization changes how the paper should talk about identity. "Adam" is not a free-floating persona narrative layered over a generic base model. It is a runtime identity maintained by the repeated interplay between graph state, retrieval assembly, operator intervention, and attributable observability. The model contributes generation. The persistence substrate carries continuity. The membrane and feedback surfaces enforce operator-visible constraint. The observatory makes parts of that continuity inspectable and, in live mode, perturbable under explicit provenance. The result is an architecture in which identity is less a hidden internal latent and more a maintained relation among persisted artifacts.

The third architectural fact is that selection is split across two temporal scales. Regard and graph updates operate at the durable scale. The active set, inference profile, and prompt budget operate at the turn scale. This is why the paper must not collapse regard into salience. Adam's current code makes room for an item to be persistent without being selected in the current turn, and conversely for a turn to privilege something newly spoken by the operator without granting it durable selection pressure. That split is crucial for a system that wants to speak about "memory" or "persona" without making those words mystical. Persistent selection is graph-mediated and event-mediated. Turn-time assembly is aperture-limited and budgeted.

The repo's current inference layer sharpens that claim further. `manual`, `runtime_auto`, and `adam_auto` are the current declared inference modes, but only two effective families are real on the MLX path in practice: manual and runtime-auto. On MLX, `adam_auto` is currently a bounded contract wrapper that drops to runtime-auto rather than invoking a hidden persona chooser or recursive prepass.[28] This is why the public paper should not imply a hidden chooser, hidden policy network, or governor-style deliberation layer. The repo gives the writer a simpler but more defensible claim: Adam v1 exposes an explicit bounded inference-profile system whose fallbacks are visible and logged.

The membrane then closes the loop between raw generation and operator-facing response. In older conceptual language it was easy to talk as if Adam's response were the model's response simpliciter. The current repo is more exact. The model emits raw text, the runtime sanitizes it,

[19] The normative turn order is `docs/TURN_LOOP_AND_MEMBRANE.md:L7-L12`. The runtime entry points that carry this story are `eden/runtime.py:L429-L452` for preview, `eden/runtime.py:L2426-L2474` for chat, `eden/runtime.py:L2069-L2152` for sanitization, and `eden/runtime.py:L2624-L2742` for feedback application.

[20] The scope boundary is explicit in `docs/PROJECT_CHARTER.md:L16-L19`. The status surface reiterates "Weight training / fine-tuning / LoRA" and "Governor / hidden planner" as absent or deferred in `docs/IMPLEMENTATION_TRUTH_TABLE.md:L90-L92`.

[21] `docs/KNOWN_LIMITATIONS.md:L5-L6`; `eden/inference.py:L295-L314`.

[22] The current log shows this concretely at `logs/runtime.jsonl:L166-L167`.

[23] `eden/inference.py:L305-L314`; `logs/runtime.jsonl:L166-L167`.

[24] `docs/KNOWN_LIMITATIONS.md:L19-L25,L48-L54`.

[25] The externalized persistence claim is reflected in the graph schema and turn loop contracts: `docs/GRAPH_SCHEMA.md:L82-L91`; `docs/TURN_LOOP_AND_MEMBRANE.md:L57-L71`.

[26] `tests/test_runtime_e2e.py:L11-L50`.

[27] Figure 8 summarizes selected current counts from `data/eden.db`; Figure 7 aggregates the runtime log.

[28] `eden/inference.py:L10-L18,L177-L186,L295-L340`.

and the operator-facing response is the membrane result. This matters because a major portion of the runtime's honesty contract is enforced there. The current e2e test proves that scaffold headings such as "Answer:", "Basis:", and "Next Step:" are absent from the persisted membrane text.[29] The hum tests add a complementary guarantee: the hum artifacts and their file paths do not get injected back into the subsequent prompt assembly as a hidden continuity channel.[30] The architectural meaning of these tests is that Adam's current continuity story is not a covert prompt-smuggling mechanism. It is graph-backed continuity plus explicit, inspectable auxiliary artifacts.

All of this is compatible with calling Adam a graph-conditioned runtime, provided the phrase is defined carefully. "Graph-conditioned" does not mean that every response is determined by some monolithic graph traversal algorithm, nor that layout geometry itself proves meaning. It means that the persistent graph shapes what can be retrieved, how continuity is maintained, what feedback has lasting force, and what the observatory can measure or mutate with provenance. That is a robust claim under current repo evidence. A stronger claim that the graph alone constitutes Adam's entirety would be a philosophical overreach. The current repo supports a more modest but still consequential sentence: Adam's continuity and observability are materially graph-mediated.

4 Primary Runtime Surface and Secondary Measurement Surface

The current project charter says plainly that the TUI is the primary runtime instrument. That is not a merely nostalgic preference for terminal aesthetics. It is an architectural boundary about where dialogue actually occurs, where the direct loop is centered, and what the browser is supposed to do.[31] The README and TUI spec reinforce that boundary by describing a dialogue-first layout in which the live chat session, composer, action shelf, and runtime telemetry are brought together in one bounded shell rather than scattered across a launcher-plus-browser hybrid.[32] The TUI therefore carries not only operator input but the immediate presentation of active state, reasoning-adjacent surfaces, and runtime signals. If the paper were to describe the browser as Adam's main interface, it would be describing a different system than the one the repo presently implements.

This does not make the observatory secondary in the sense of being trivial. The current observatory is one of the strongest parts of the repo, and the current docs and audit surfaces are unusually explicit about its contract.

The observatory serves three families of function. First, it is a read surface: graph, basin, geometry, measurement ledger, transcript/runtime trace, and Tanakh sidecar payloads can all be exposed there, with live and static modes disclosed honestly.[33] Second, it is a comparison and preview surface: the browser can render baseline versus modified state through `preview_graph_patch`, compare selections, and local layout modes without silently mutating the graph.[34] Third, under the live API contract, it is a bounded mutation surface: preview, commit, and revert are available as attributable actions that feed the measurement ledger and runtime trace.[35]

The authority boundary is where most public explanations go wrong, so it deserves precision. The browser renders both authoritative and non-authoritative state. Layout presets, preview styling, coordinate choices, filter presets, table sorting, and workspace arrangement are intentionally browser-local view state only.[36] They can shape what the operator sees, but they are never part of the measurement ledger. By contrast, a committed graph edit, memode assertion, or revert action creates a persistent measurement event and can trigger graph, measurement, and runtime-trace refreshes.[37] In other words, the observatory contains both appearance state and evidence-bearing mutation state, but it marks the boundary between them explicitly. The public paper must preserve that distinction or it will overclaim browser authority.

The current browser code supports that claim rather than undermining it. In the React app, mutation capability is guarded by a `canMutate` check that requires live mode plus preview and commit endpoints in the bootstrap payload.[38] In static mode, the action chrome remains visible, but the UI says that preview/commit/revert are visible yet disabled and that mutation remains live-only.[39] This is an important design honesty move. Instead of hiding mutation affordances when static exports are loaded, the repo keeps the contract legible and mode-bound. The user can see what the live workbench can do without being tricked into believing that a static export is authoritative.

The checked-in browser audit surfaces extend that story from specification to broader evidence. The e2e audit says that current Chromium journeys J10 through J14 prove browser-visible measure preview, attributable edge edit, memode assertion and update, explicit revert, and runtime-trace causality under the live preview/commit/revert

[29] `tests/test_runtime_e2e.py:L19-L33`.

[30] `tests/test_hum_runtime.py:L92-L121`.

[31] `docs/PROJECT_CHARTER.md:L18-L19`.

[32] `README.md:L63-L65,L123-L142`; `docs/TUI_SPEC.md:L150-L164`.

[33] Read-surface scope is described in `docs/OBSERVATORY_SPEC.md:L66-L116,L187-L216`.

[34] `docs/OBSERVATORY_SPEC.md:L148-L178,L199-L209`; `docs/OBSERVATORY_INTERACTION_SPEC.md:L31-L53`.

[35] `docs/OBSERVATORY_SPEC.md:L320-L343`; `docs/MEASUREMENT_EVENT_MODEL.md:L46-L57`.

[36] `docs/OBSERVATORY_INTERACTION_SPEC.md:L138-L147`; `docs/OBSERVATORY_SPEC.md:L176-L178,L216-L216`.

[37] `docs/MEASUREMENT_EVENT_MODEL.md:L51-L57`.

[38] `web/observatory/src/App.tsx:L853-L871`.

[39] `web/observatory/src/App.tsx:L2258-L2270,L3868-L3870`.

API.[40] The same audit also records the static-mode limits: direct `file://` opening is unsupported, static exports should be HTTP-served, and static mode must not masquerade as live authority.[41] The current Step 3 run refreshed server-side measurement tests and the build-meta check rather than rerunning the full Playwright ladder, so the paper keeps its browser claims bounded to what the code, docs, and existing checked-in browser audit actually sustain.

This is the right place to define the current browser contract gap precisely. A contract gap exists whenever the server-side or documented mechanism is real but current browser exposure is limited by mode, audit freshness, or current implementation surface. In Adam v1 the strongest contract gap is not that preview/commit/revert are absent. They are present server-side and checked in as browser journeys. The stronger gap is that those powers are mode-bound and deliberately non-universal: static exports remain read surfaces, runtime trace is live-only, and view-state changes never become evidence. The gap is therefore not a missing feature list but a disciplined authority boundary. The paper can say that the observatory is bidirectional under the live API. It cannot say that any browser render of the observatory is a full graph IDE or a universally writable control plane.

The measurement ledger is the cleanest place where the observatory's constructive role becomes visible. In the current export family, the measurement ledger is not a vague audit note. It is a structured persistence surface that records action type, operator and evidence labels, before-state, proposed-state, committed-state, and reversion linkage.[42] Reversion is itself a new event. That means the system does not achieve reversibility by silently mutating history out of existence. It achieves reversibility by persisting another attributable step in the same ledger family. This is a stronger scientific and operational design than a purely imperative "undo" button because it keeps the history of intervention inspectable.

The observatory's geometry surfaces deserve similar care. The current build exports geometry diagnostics, local reports, and coordinate methods, but it does not let layout aesthetics become evidence by default. The geometry layer explicitly labels some metrics as observed and others as derived.[43] That explicit labeling is why the paper can discuss geometry as instrumentation without claiming that a visually coherent layout proves memetic structure. Geometry is useful because it makes particular graph properties inspectable and because it can be ablated, compared, and exported. It is not useful as a replacement for ontological or evidentiary discipline.

5 Observed Operational Evidence

The strongest current evidence for Adam's behavior comes from three mutually reinforcing sources: targeted tests refreshed in this run, persistent runtime traces already present in `logs/runtime.jsonl`, and export artifacts under `exports/` that preserve graph, geometry, measurement, and conversation surfaces. This section keeps those three layers separate. Tests prove bounded behaviors under controlled harness conditions. Runtime logs prove that the deployed repo has actually emitted relevant events. Exports prove that the measurement and observatory surfaces are material rather than merely specified.

The current Step 3 pass refreshed four execution anchors directly. First, `tests/test_runtime_e2e.py` passed in the repo-local virtual environment, refreshing the direct-loop and export-family proof surface. Second, `tests/test_observatory_measurements.py` passed, refreshing the server-side preview/commit/revert and memode-update persistence surfaces. Third, `tests/test_hum_runtime.py` passed, refreshing the bounded hum and no-prompt-injection surfaces. Fourth, `scripts/check_observatory_build_meta.py` returned an `ok = true` build-meta result for the checked-in observatory bundle.[44] These are bounded proofs, not global guarantees, but they are current-turn evidence and therefore deserve more weight than inherited summaries.

The first completed observed episode is the direct runtime loop. In `tests/test_runtime_e2e.py::test_single_graph_bootstrap_chat_feedback_and_exports`, the harness initializes a blank experiment, starts a session, asks "How does Adam remain the same persona over time?", and then checks that a turn was persisted, that meme counts increased, that the active set is non-empty, that the requested inference mode remained manual, and that the membrane text excludes scaffold headings.[45] It then applies edit feedback with corrected text, performs a retrieval query that must surface the feedback-derived material, and exports the whole experiment, asserting the presence of graph, basin, geometry, measurement ledger, observatory index, and Tanakh sidecar outputs.[46] As a single bounded episode, this is quite dense. It shows that persistent graph growth, membrane sanitation, explicit feedback, later retrieval pressure, and the export family all belong to one operational loop rather than to disjoint subsystems.

[40] `docs/OBSERVATORY_E2E_AUDIT.md:L15-L16,L143-L193`.

[41] `docs/OBSERVATORY_E2E_AUDIT.md:L217-L231`; `docs/KNOWN_LIMITATIONS.md:L53-L54`.

[42] See Figure 5 and the underlying export at `exports/b178bed2-731e-4f8b-b5f8-a93d1300b2f7/measurement_events.json`.

[43] `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/geometry_diagnostics.json`; Figure 6.

[44] Current-run execution results: `./.venv/bin/pytest-qtests/test_runtime_e2e.py` produced `14 passed`; `./.venv/bin/pytest-qtests/test_observatory_measurements.py` produced `12 passed`; `./.venv/bin/pytest-qtests/test_hum_runtime.py` produced `4 passed`; `./.venv/bin/pythonscripts/check_observatory_build_meta.py` returned `{"ok": true}`.

[45] `tests/test_runtime_e2e.py:L11-L33`.

[46] `tests/test_runtime_e2e.py:L35-L75`.

The current runtime log corroborates that the repo does in fact emit the kinds of events that the test harness reasons over. For example, a manual MLX generation on experiment `bb298723-...` is recorded as a turn preview, a generation start, a generation complete event, and then a hum refresh that references the resulting turn and membrane-event count.[47] That sequence matters because it shows the temporal ordering between prompt assembly, model completion, and hum refresh in a live trace rather than just in code. The runtime event histogram in Figure 7 adds a second level of corroboration: export and generation surfaces dominate the current trace family, which is exactly what one would expect in a repo whose strongest operational evidence is trace-heavy and export-heavy rather than benchmark-heavy.

The second completed observed episode is observatory mutation with explicit reversion. The current server-side measurement test seeds a session, previews an `edge_add` action, commits it, proves that the new edge exists, then reverts the committed event and proves that the edge has disappeared again.[48] The exported measurement ledger from experiment `b178bed2-...` is fully consistent with that contract. It records one committed `edge_add` event and one distinct `revert` event, each with action type, operator label, evidence label, before-state, proposed-state, committed-state, and event linkage.[49] The runtime log then adds a third evidence layer: lines 109 through 114 record the `observatory_commit` and `observatory_revert` events for that same experiment and session.[50] Taken together, these three surfaces support a strong but still bounded claim: live observatory mutation and explicit reversion are not merely described; they are implemented, persisted, and trace-visible.

The third completed observed episode is hum refresh without prompt injection. In `tests/test_hum_runtime.py::test_feedback_refreshes_hum_without_prompt_injection`, the harness runs a chat turn, applies edit feedback, loads the hum payload, then asks the runtime for the next-turn preview and concatenates the system prompt, conversation prompt, history context, and feedback context.[51] The test proves two things at once. First, the hum payload reflects feedback presence and count. Second, the prompt-assembly surfaces do not contain the hum artifact file paths, which would indicate covert prompt injection. The live transcript from session `8a5213f7-...` corroborates the positive side of that story: it shows a concrete Adam turn, a feedback entry, and an explicit hum presence section with markdown and JSON paths.[52] The runtime log corroborates the timing side: generation completion is followed immediately by `hum_refreshed` on the same session and turn.[53] The safest implementation-strength sentence is therefore that the hum is a bounded continuity artifact refreshed from persistent surfaces and referenced by session snapshots and transcripts, but not injected back into prompt assembly as a hidden generation channel.

Beyond these three episodes, the export family shows the scale and composition of Adam's current persistent state. The current SQLite store contains 82,196 meme rows, 18,383 memodes, 447,516 edges, 75 turns, 53 feedback events, two measurement events, 75 membrane events, and 240 trace events.[54] The graph export for experiment `bb298723-...` reports 28 sessions, 74 turns, 8,285 memes, 1,901 memodes, 48,279 edges, 53 feedback items, and two measurement events in the scoped export family.[55] These numbers do not by themselves prove quality, but they do prove that Adam's persistence substrate is non-trivial and that its scoped export surfaces can differ from global store counts. That scoped/global distinction is another place where the paper needs discipline. It should speak of exported slices and global store state as related but non-identical evidence surfaces.

The geometry export adds a comparable scale signal while also illustrating the repo's evidence labeling discipline. On the full graph of experiment `bb298723-...`, the export reports observed metrics such as circularity, radiality, linearity, community structure, and triadic closure, while also reporting derived metrics such as mirror symmetry, chirality, and translation symmetry.[56] The public paper can therefore discuss the presence of a geometry instrument and can mention specific exported scores, but it should not elevate derived symmetry proxies into proof of cognitive organization. The repo itself has already done the hard work of labeling those surfaces honestly; the paper's job is to keep that honesty intact.

One important caution emerges from the export family as well. The `observatory_index.json` summary for experiment `bb298723-...` reports a measurement-event count of zero even though the corresponding graph export reports two measurement events.[57] The right interpretation is not that one file is "wrong" in some global sense. The stronger and more careful interpretation is that export freshness and scope can diverge across surfaces and that the paper must not silently harmonize them. This is precisely the kind of bounded inconsistency a scientific whitepaper should surface rather than hide, because it marks a real closure task: the observatory summary and

[47] `logs/runtime.jsonl:L1232-L1235`.

[48] `tests/test_observatory_measurements.py:L53-L110`.

[49] `exports/b178bed2-731e-4f8b-b5f8-a93d1300b2f7/measurement_events.json`; Figure 5.

[50] `logs/runtime.jsonl:L109-L114`.

[51] `tests/test_hum_runtime.py:L92-L121`.

[52] `exports/conversations/adam-graph-bb298723/operator-session-8a5213f7.md:L10-L32`.

[53] `logs/runtime.jsonl:L1233-L1235`.

[54] Figure 8; underlying counts from `data/eden.db`.

[55] `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/graph_knowledge_base.json`; Figure 3.

[56] `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/geometry_diagnostics.json`; Figure 6.

[57] `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/observatory_index.json`; `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/graph_knowledge_base.json`.

scoped graph exports need a clearer freshness invariant if public readers are to treat them as interchangeable snapshots.

6 Archaeology, Drift, and Whitepaper Refactoring

The current run's designated baseline manuscript is `assets/white_paper_pipeline/white_paper_drafts/eden_whitepaper_v14.pdf`. Because the current prompt explicitly elevates that draft as the baseline under revision, it is not treated here as merely historical noise. It is the prose surface against which this whitepaper had to perform its strongest refactoring. The key archaeological result is straightforward: the baseline still carries implementation claims that the current Adam repo no longer supports. Figure 1 makes the simplest version of that point numerically by showing that terms such as "governor" and "recursive" are concentrated in the baseline far more than in the current README, project charter, intelligence brief, or pre-writing memo. The content difference is not cosmetic. It marks a change in what the repo currently claims to implement.

The most important vanished mechanism is the governor-first turn architecture. The baseline says, in effect, that every turn begins with a governor plan, that a governor persona and JSON planning surface stand between operator input and Adam response, and that recursive language-model decomposition still has an active architectural role. The current repo does not support those claims. The current charter says "No governor in v1." The current truth table marks governor and hidden planner surfaces as absent or deferred. The current inference layer shows `adam_auto` falling back to `runtime_auto` rather than invoking a hidden prepass chooser. The turn-loop docs specify a direct loop with retrieve/assemble, response, membrane, feedback, and graph update.[58] The correct editorial action is therefore not to "soften" governor language slightly. It is to tombstone it as a current architectural description and replace it with the direct loop the repo actually sustains.

The second major drift surface concerns ontology. The baseline leans on a looser idea of a unified meme/memode substrate in which memodes can read as little more than higher-order names or recursive behavioral bundles. The current ontology is much tighter. It makes information explicit as a projected knowledge unit, keeps knowledge relations separate from memode support, refuses to auto-promote clusters into memodes, and defines the memode admissibility floor in graph-theoretic rather than merely conceptual terms.[59] The public-writing implication is that older sentences about memodes surviving as free-floating identities, or about cluster-like structures functioning as memodes simply because they are semantically coherent, are now overclaims. The current paper must preserve the memetic ambition of the project while binding it to the stricter object model that the repo currently implements.

The third drift surface is evidentiary rather than ontological. The baseline relies on evidence families and file paths that the current repo no longer centers, including older log naming and older orchestration modules. The present repo has stronger and more numerous evidence surfaces than that draft acknowledged, but they are different surfaces: `logs/runtime.jsonl`, conversation markdown exports, scoped observatory JSON and HTML artifacts, measurement ledgers, geometry diagnostics, memode audits, and current-turn tests. The whitepaper's empirical layer therefore had to be rebuilt around those current evidence substrates rather than inheriting older references.[60] This is not a mere modernization of filenames. It changes what kinds of claims can be made publicly, because the current evidence is more trace-driven, more export-driven, and more mode-explicit.

Not everything in the baseline had to be discarded. Several continuities remain admissible and were preserved. The project is still local-first. The graph is still central to persistence. Feedback still has explicit force rather than being inferred silently. The system still uses memetic vocabulary to talk about durable behavioral structure. The observability ambition is still present, and indeed is stronger in current code than in earlier prose because the observatory now has a clearer mutation and measurement contract. The critical point is that continuity now survives under tighter denotation. "Local-first" is not a generic slogan but a concrete MLX-plus-local-store fact. "Memode" is not a poetic label but a derived second-order object with support-edge criteria. "Observability" is not passive charting but a live or static browser workbench with explicit authority boundaries.

The result is a more austere but also more defensible public artifact. Older prose could sound grander because it described mechanisms that were only partially constrained by current code. The current repo supports something more interesting scientifically: a system willing to expose its own boundedness. Adam v1 is not a hidden-planner system pretending to be transparent. It is a graph-conditioned runtime that makes much of its operative persistence and intervention structure inspectable. It can therefore be described with fewer dramatic claims and more operational clarity.

This archaeological refactoring also changes how future

[58] `docs/PROJECT_CHARTER.md:L16-L19`; `docs/IMPLEMENTATION_TRUTH_TABLE.md:L90-L92`; `docs/TURN_LOOP_AND_MEMBRANE.md:L7-L12`; `eden/inference.py:L295-L314`.

[59] `docs/CANONICAL_ONTOLOGY.md:L10-L24,L51-L57`.

[60] Current evidence-family examples include `logs/runtime.jsonl`, `exports/conversations/adam-graph-bb298723/operator-session-8a5213f7.md`, `exports/b178bed2-731e-4f8b-b5f8-a93d1300b2f7/measurement_events.json`, and `exports/bb298723-5fbf-4554-bf6b-ec5f4d336fbd/graph_knowledge_base.json`.

writing should proceed. Whitepaper maintenance for Adam cannot be a simple version-bump process in which older claims are assumed to persist unless directly contradicted. The repo's own patch manifests, briefs, memos, and exports are drift surfaces. Claims can vanish, tighten, or move. Definitions can survive rhetorically while changing denotation. Browser capability can expand server-side before it becomes uniformly browser-exposed. The correct whitepaper workflow is therefore recursive in a disciplined sense: each new prose pass must recompile the current repo and its current evidence rather than trusting lineage alone.[61]

7 Limitations, Contract Gaps, and Closure Work

Adam v1 is substantial enough to warrant a real whitepaper, but it is not complete in every direction that the repo's older conceptual orbit once implied. The first limitation is explicit in the charter and status surfaces: there is no governor, hidden planner, fine-tuning loop, LoRA persistence path, or embedding-based semantic retrieval in the current build.[62] These are not implementation omissions that the paper should euphemize as "under the hood." They are absent or deferred surfaces. Future work can revisit them, but present-tense public claims should not.

The second limitation concerns the observatory. The live browser workbench is materially strong, but it remains a bounded observability and measurement instrument rather than a fully general graph IDE with arbitrary import, unconstrained drag authoring, or mode-free mutation authority.[63] Static mode remains read-only. Direct `file://` opening is not a supported path. Runtime trace is live-only. Some browser claims rely on checked-in e2e audit surfaces rather than on a fresh Step 3 Playwright rerun. The whitepaper can still describe the browser as a bidirectional measurement workbench, because the server contract, browser code, tests, and checked-in audit surfaces together sustain that sentence. What it cannot do is imply that every render of the observatory is equally authoritative or that browser-local state carries evidentiary force.

The third limitation concerns geometry and export freshness. The repo already warns that large graphs may use sparse-safe approximations rather than dense exact layouts and that observatory or export freshness can vary by artifact family.[64] The mismatch between measurement counts in the current `observatory_index.json` and the scoped graph export is one concrete example. The correct future-work response is not to retreat from geometry or export claims entirely. It is to tighten freshness and scope invariants so that public readers can know which surfaces are meant to be directly comparable and which are not.

The fourth limitation is interpretive. Adam uses memetic language, but its strongest currently observed memetic evidence is structural rather than yet fully metricized in terms of fidelity, fecundity, and longevity. The memode audit proves that derived assemblies, support edges, and informational relations are currently separable. Regard proves that some form of durable selection pressure exists. But the repo does not yet emit a mature family of explicit memetic proxy metrics. That is why this run includes synthetic figures for such proxies only as pipeline demonstrators rather than observed results. The paper should be honest that current memetic language is operationally grounded but not yet fully instrumented at every theoretical dimension it might someday support.

The fifth limitation is narrative rather than mechanical. Because the repo still contains legacy `EDEN` strings, old seed-canon references, and historical conceptual anchors, any public Adam paper risks inheriting stale denotation if it is not actively policed. The present paper has tried to solve that by naming the legacy material once, using it archaeologically, and then re-centering Adam v1. Future public writing will need the same discipline. Otherwise the project will drift back into a mixed shell/runtime story that obscures the sharper technical object the current repo now supports.

The shortest closure path from the current state is concrete. First, refresh and unify export freshness invariants so that high-level observatory summaries and scoped graph exports cannot silently disagree about measurement counts. Second, extend current-turn empirical practice from targeted tests plus existing logs toward a small durable evaluation harness that can emit the same evidence families under fixed conditions. Third, if memetic proxy language will remain load-bearing, add explicit metrics for durability, recurrence, and spread rather than leaving those terms only partially operationalized. Fourth, preserve the browser authority boundary even if the workbench grows; new affordances should strengthen provenance rather than blur it.

8 Conclusion

Adam v1 is best understood not as an aspirational umbrella for every historically adjacent idea in the repository, but as the concrete runtime that the current repo actually implements. That runtime is local-first, graph-

[61] The archaeology surfaces used in this run include `docs/PATCH_MANIFEST_V1_1.md`, `docs/PATCH_MANIFEST_V1_2.md`, `docs/MIGRATION_NOTES_V1_1.md`, the selected intelligence brief, the selected pre-writing memo, the baseline draft, and `codex_notes_garden.md`.

[62] `docs/PROJECT_CHARTER.md:L16-L19,L35-L36`; `docs/KNOWN_LIMITATIONS.md:L43-L45`.

[63] `docs/KNOWN_LIMITATIONS.md:L21-L25,L48-L54`.

[64] `docs/KNOWN_LIMITATIONS.md:L19-L25,L53-L54`.

conditioned, event-rich, and unusually explicit about its own measurement boundaries. It externalizes persistence into graph state and explicit feedback instead of model-weight adaptation. It treats memes as first-class persistent units and memodes as derived second-order structures with an admissibility floor. It uses regard as durable selection pressure rather than collapsing selection into momentary salience. It keeps the membrane visible and attributable. It centers the TUI for live dialogue while using the browser observatory as a secondary measurement and mutation workbench with explicit live/static boundaries.

Those are significant achievements, and they are stronger precisely because the paper has refused to overclaim beyond them. The current repo does not need a fictive governor, a hidden planner, or a retrofitted recursive-language-model mythology to be interesting. Its stronger claim is that it can sustain a public technical description under evidence pressure. That is the standard this whitepaper has attempted to meet. Where the repository implements a mechanism, the paper says so. Where it instruments without full proof, the paper names instrumentation. Where earlier prose outruns current implementation, the paper downgrades or tombstones the claim instead of smuggling it forward. If Adam is to remain scientifically legible as it evolves, that discipline is not optional metadata. It is part of the system.

A Appendix A: Statement Lineage Notes

Three classes of statement recur across the repo lineage and needed explicit disposition in this revision. First, the sentence family claiming that every turn begins with a governor plan is tombstoned in the main body because current code, tests, and docs no longer support it. Second, statement families claiming that memodes are durable identity carriers in a broad or cluster-like sense are preserved only after tightening their denotation to the current admissibility-gated derived object model. Third, statement families describing observation as constructive rather than merely descriptive are preserved, because the live preview/commit/revert contract and measurement ledger now materially support them. In short: the paper preserves the project's strongest recurring statements only where their present denotation survives current repo truth.

B Appendix B: Local Evidence Notes

The strongest locator families used throughout this paper are the current normative docs under `docs/`, the current runtime code under `eden/`, current-turn refreshed tests under `tests/`, the canonical runtime log at `logs/runtime.jsonl`, and scoped export families under `exports/`. The paper intentionally avoids laundering those local surfaces into a conventional public bibliography. Instead, they are cited as path-addressable evidence notes. Readers who want to reproduce the paper's strongest claims should begin with the following sequence: `docs/PROJECT_CHARTER.md`, `docs/CANONICAL_ONTOLOGY.md`, `docs/TURN_LOOP_AND_MEMBRANE.md`, `docs/MEASUREMENT_EVENT_MODEL.md`, `eden/inference.py`, `eden/runtime.py`, `tests/test_runtime_e2e.py`, `tests/test_observatory_measurements.py`, `tests/test_hum_runtime.py`, `logs/runtime.jsonl`, and the export families cited in Figures 5 through 4.

C Appendix C: Figure Portfolio

Observed Claim Drift Across Baseline and Current Surfaces

Term counts across the baseline draft, current README/docs, and the Step 1/2 pipeline artifacts.

	adam	eden	governor	recursive	rlm	memode	adam_auto	observatory
baseline_v14	31	124	37	18	8	167	0	4
readme	14	20	0	0	0	4	1	25
project_charter	3	2	1	0	0	1	0	0
intelligence_brief	47	142	10	3	0	23	9	122
prewriting_memo	158	36	10	2	1	12	3	57

High governor and recursive language concentration in the baseline draft, coupled with its near absence in current repo-truth surfaces, is the central drift signal driving the whitepaper rewrite.

Figure 1: [OBSERVED] Term-frequency drift across the baseline draft, current README/project charter, and the Step 1/2 pipeline artifacts. Governor and recursive language concentrate in the baseline rather than in current implementation-truth surfaces.

Observed Completed Episodes

Three bounded `DONE` episodes carried into the empirical section from current-turn execution proof.

Episode	Status	Hypothesis	Observable	Outcome
E1	DONE	The direct v1 loop persists turn state, applies edit feedback, and emits the full export family.	Turn count increments, membrane strips scaffold headings	Observed in the current test pass: `tests/test_runtime_e2e.py::test_single_graph_bootstr
E2	DONE	Preview/commit/revert persists attributable measurement events and graph rollback.	Measurement ledger delta, committed edge present after preview/revert, absent	Observed in the current test pass: `tests/test_observatory_measurements.py::test_edge_
E3	DONE	Hum refresh remains a bounded continuity artifact and stays out of prompt injection.	Hum feedback counts rise; `current_hum.md` and `current_hum.json` are absent from prompt assembly.	Observed in the current test pass: `tests/test_hum_runtime.py::test_feedback_refreshes_

These episodes are narrow: they prove current repo behavior under specific tests and traces. The paper should not generalize beyond those bounded surfaces.

Figure 2: [OBSERVED] Three bounded ‘DONE‘ episodes anchored to current-turn test execution: direct-loop e2e, observatory commit/revert persistence, and hum refresh without prompt injection.

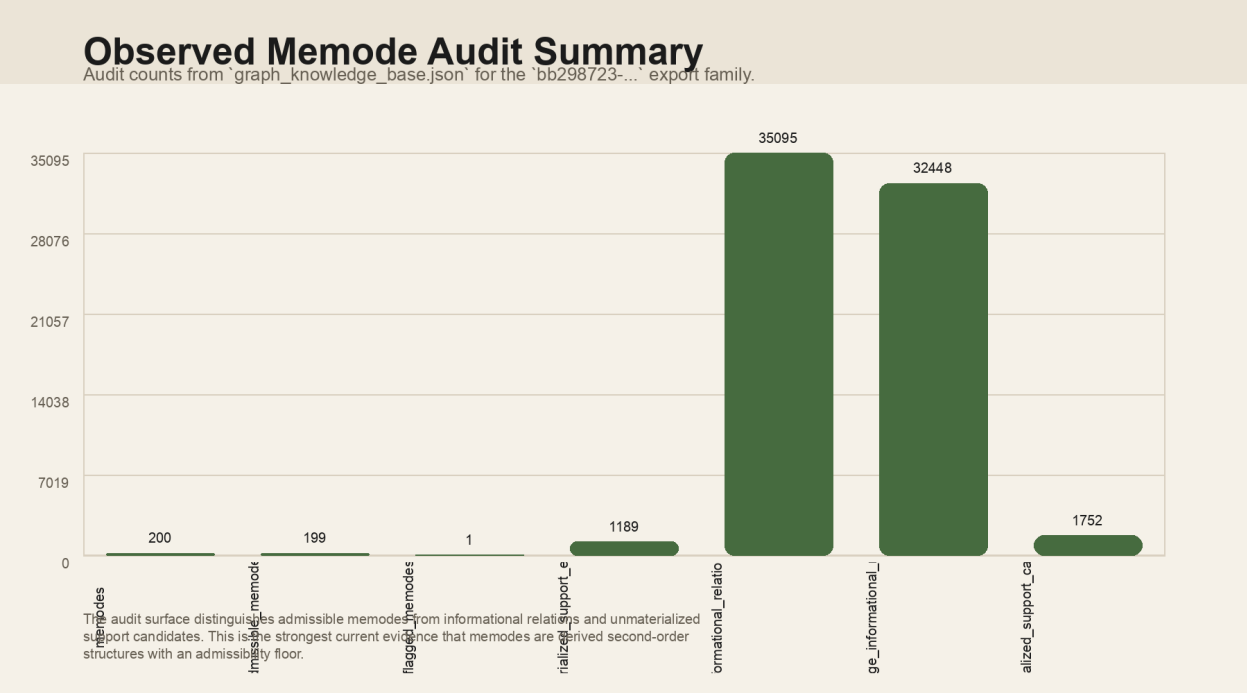

Figure 3: [OBSERVED] Memode audit summary exported with ‘graph_knowledge_base.json‘: 200 audited memodes, 199 admissible, one flagged, and 1,189 materialized support edges.

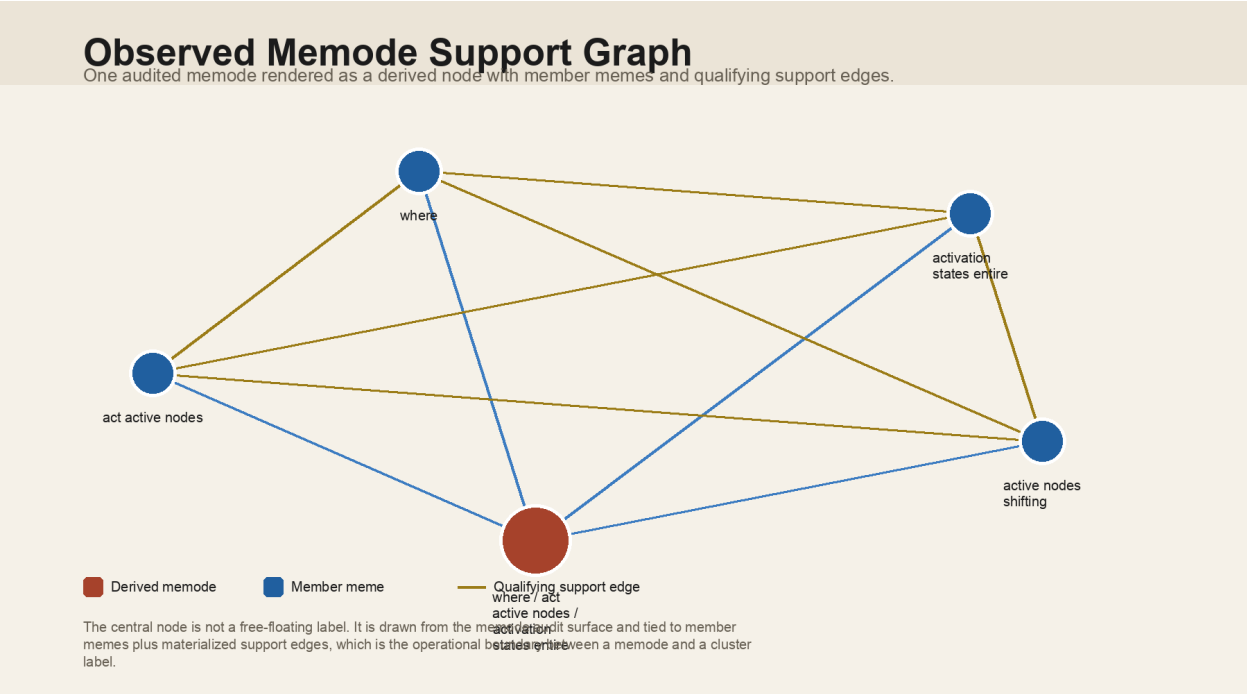

Figure 4: [OBSERVED] One audited memode from ‘graph_knowledge_base.json‘, shown with member memes and qualifying support edges. The memode is derived and admissibility-gated, not a free-floating cluster label.

Observed Measurement Ledger Events

Concrete preview/commit/revert persistence exported from the live observatory run.

Event	Action	Evidence	Created At	Reverted From
ed41de04	edge_add	OPERATOR_ASSERTED	2026-03-06 21:33:18Z	none
f689054c	revert	OPERATOR_REFINED	2026-03-06 21:33:18Z	ed41de04

The ledger records attributable graph mutation. Reversion is itself stored as a new event instead of silently rolling history backward.

Figure 5: [OBSERVED] The exported measurement ledger for experiment ‘b178bed2-...‘ shows explicit ‘edge_add‘ and ‘revert‘ events with operator and evidence metadata.

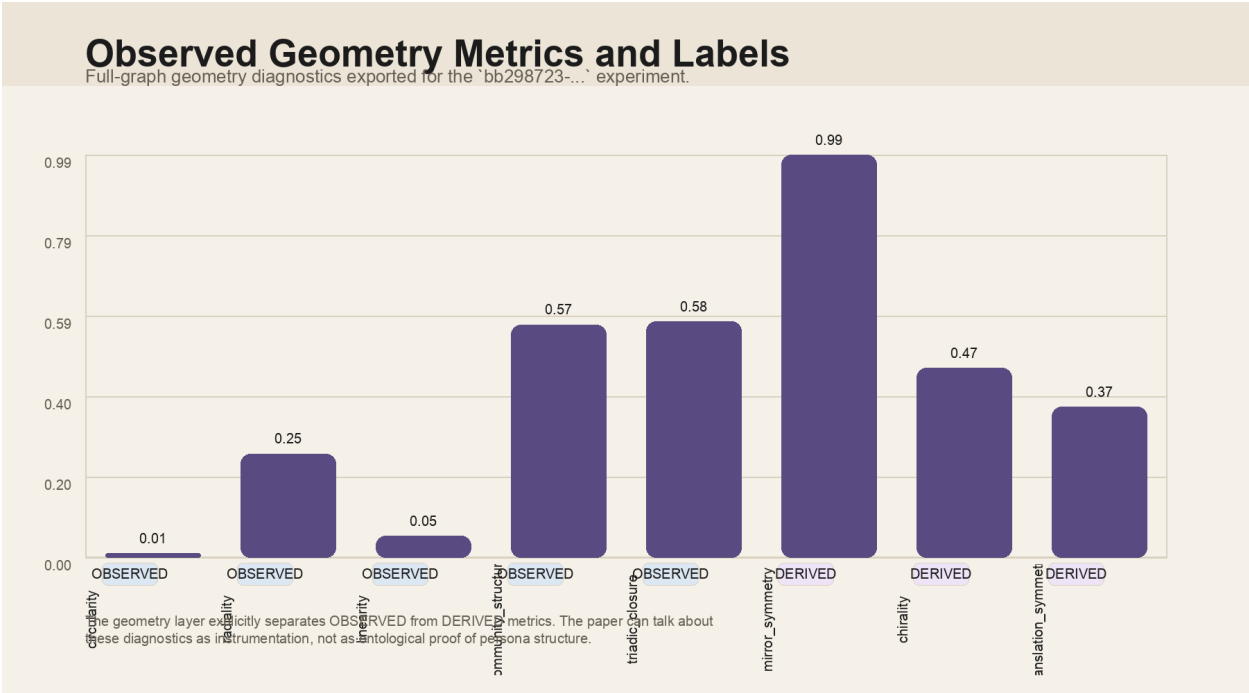

Figure 6: [OBSERVED] Geometry diagnostics exported from ‘geometry_diagnostics.json‘. The current build labels circularity, radiality, linearity, community structure, and triadic closure as observed, while symmetry proxies remain derived.

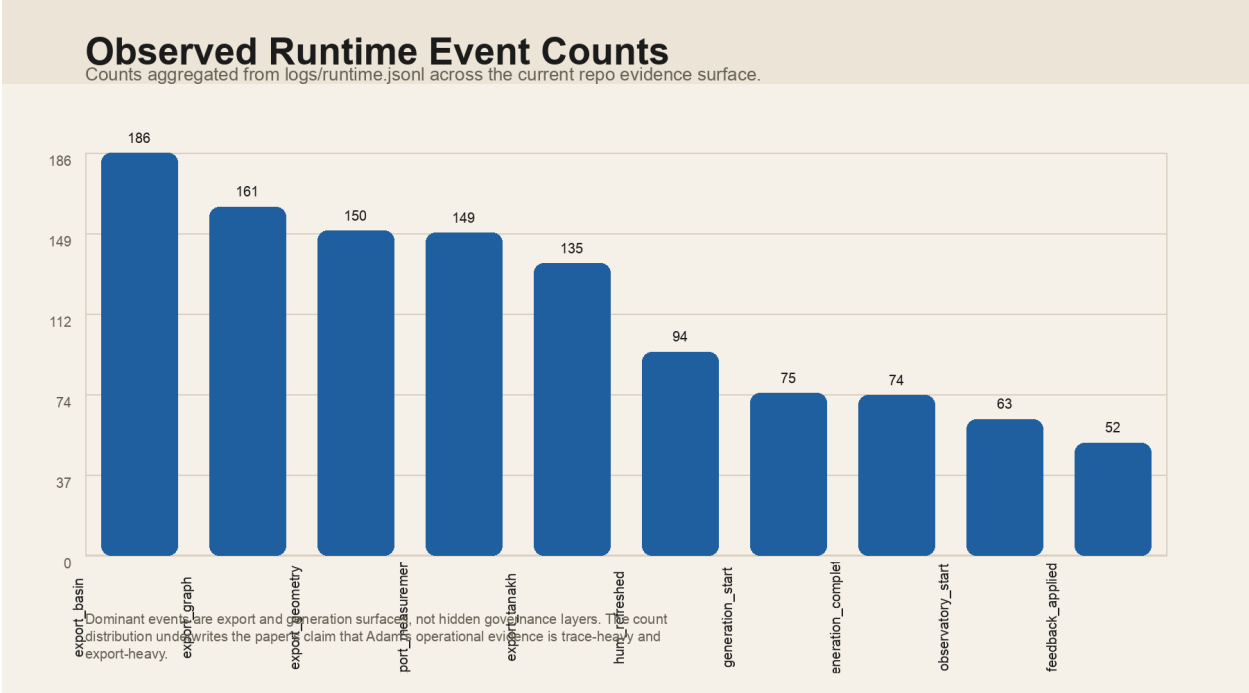

Figure 7: [OBSERVED] Runtime event counts aggregated from ‘logs/runtime.jsonl‘. Export, generation, and hum-refresh events dominate the current trace surface.

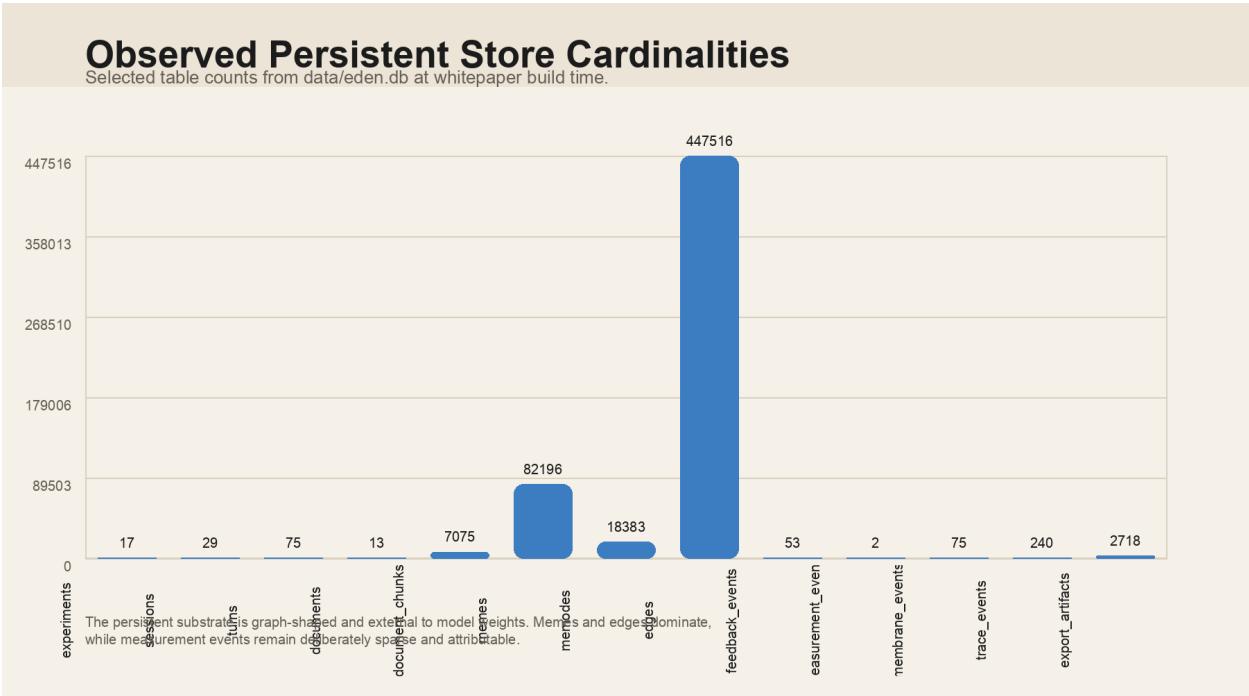

Figure 8: [OBSERVED] Selected SQLite table counts from ‘data/eden.db‘ at run time. The persistence substrate is graph state rather than model-weight adaptation.

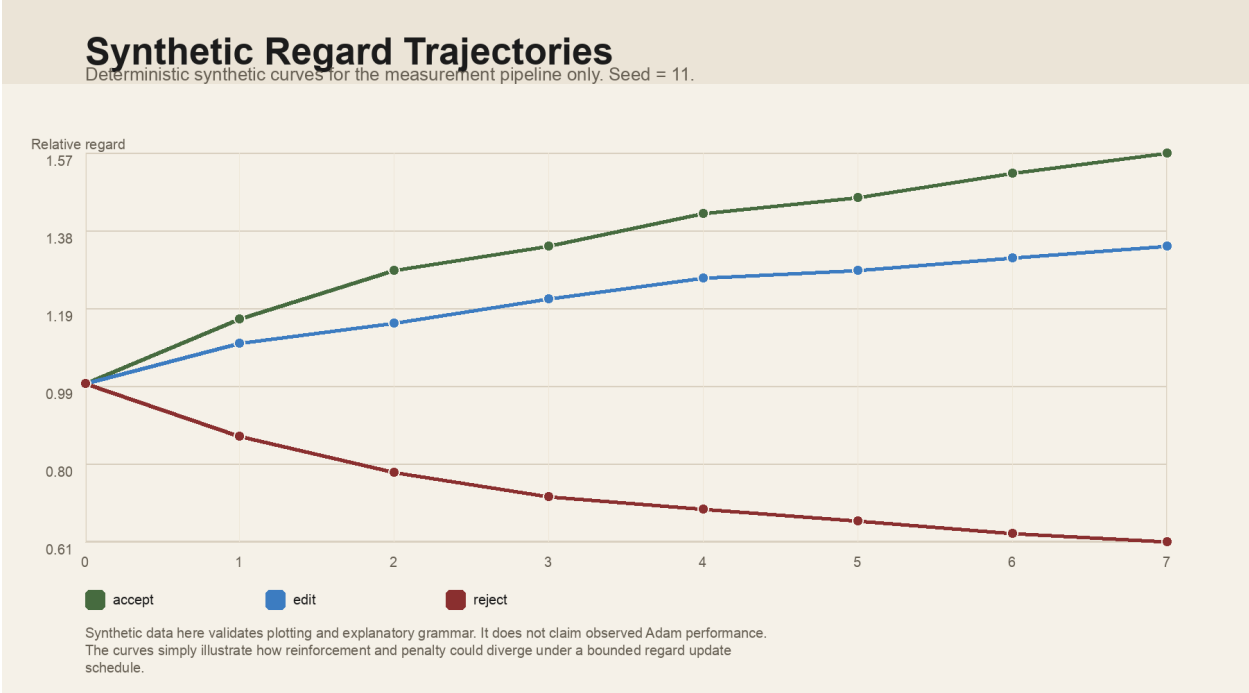

Figure 9: [SYNTHETIC] Deterministic regard trajectories used only to validate the paper's metric and visualization pipeline. Seed = 11; no observed runtime performance is implied.

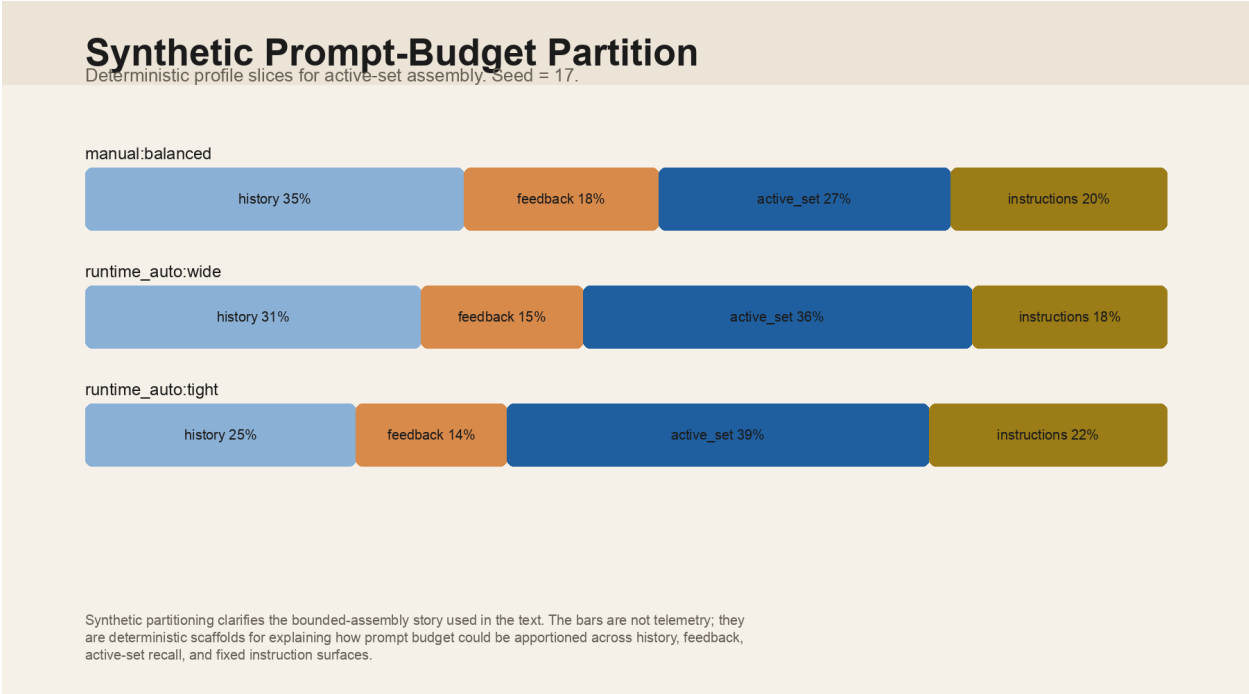

Figure 10: [SYNTHETIC] Deterministic prompt-budget partitions that illustrate bounded active-set assembly without claiming live telemetry. Seed = 17.

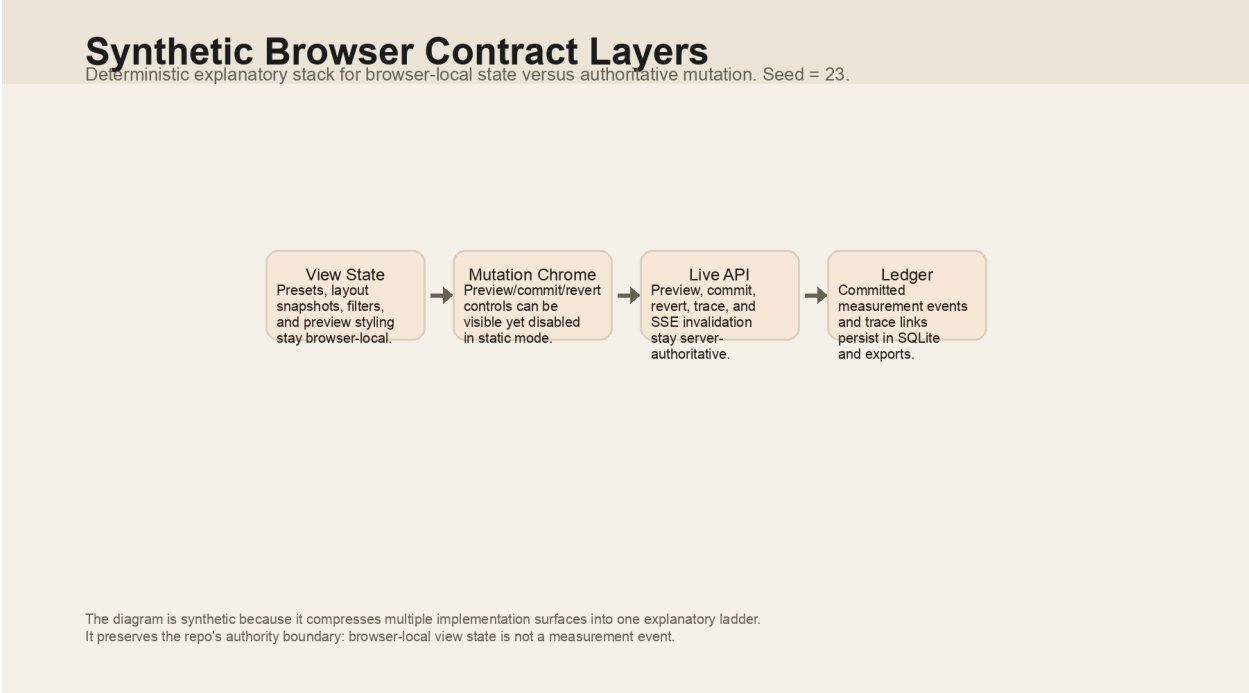

Figure 11: [SYNTHETIC] An explanatory stack that separates browser-local view state from live mutation and persistent measurement evidence. Seed = 23.

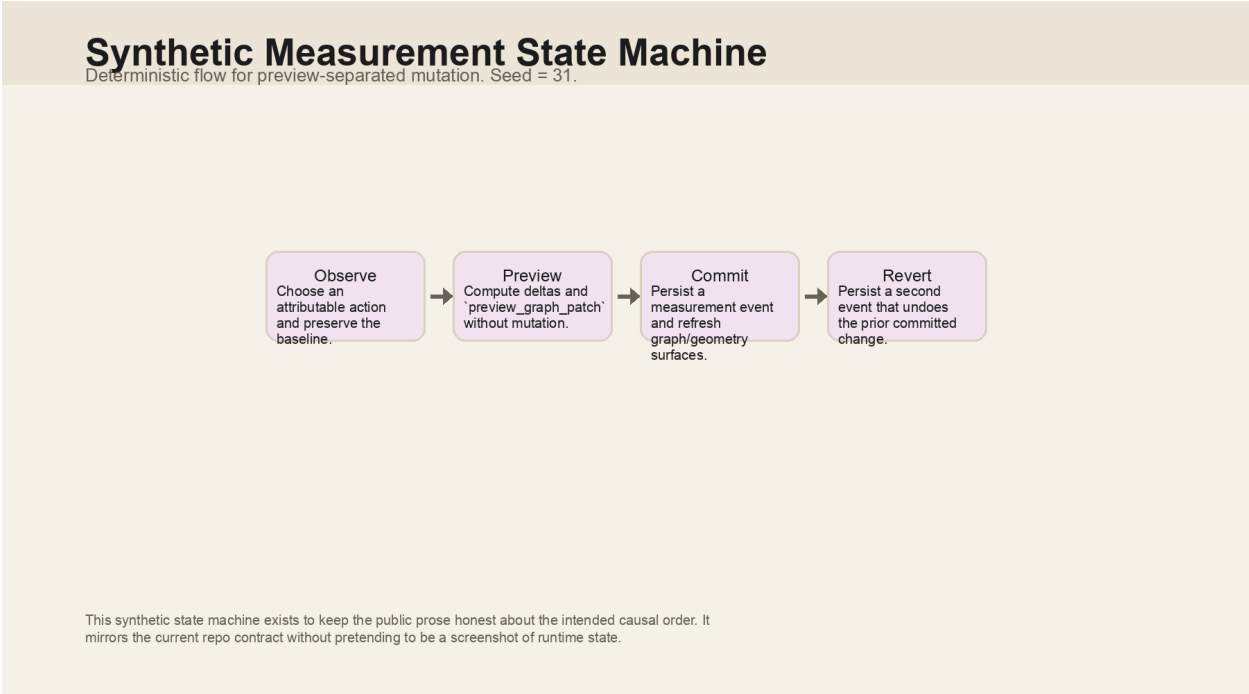

Figure 12: [SYNTHETIC] Deterministic preview-separated mutation flow used to explain the measurement-event contract. Seed = 31.

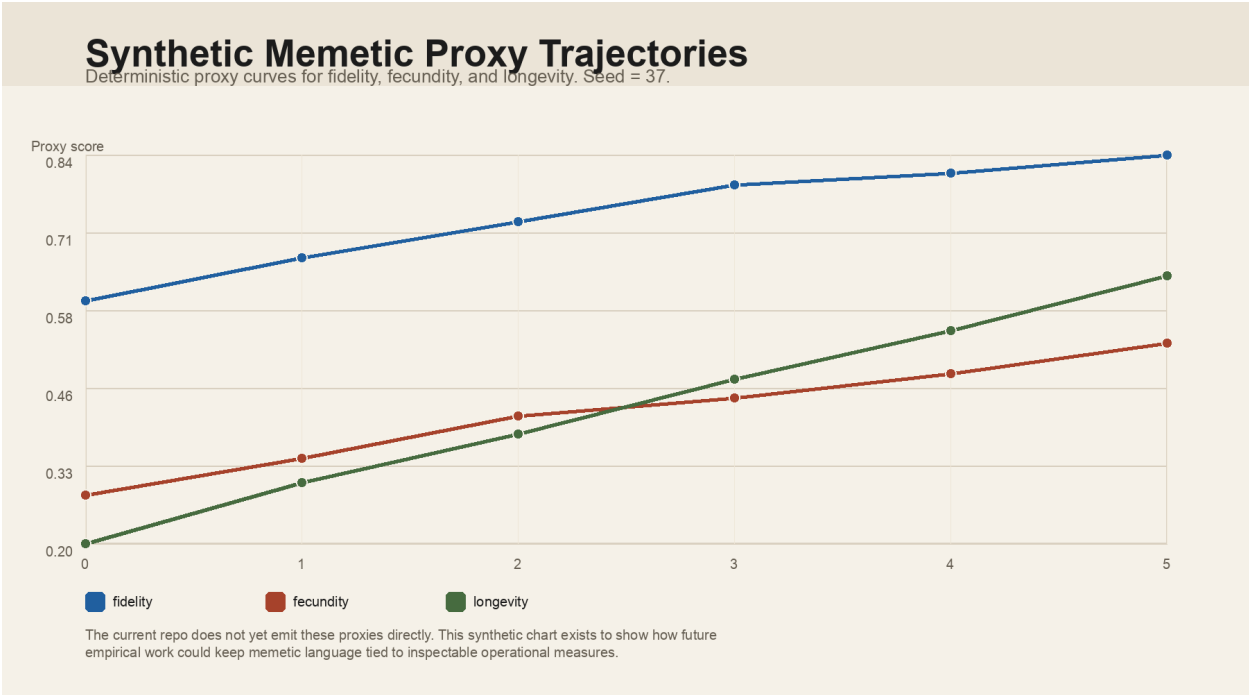

Figure 13: [SYNTHETIC] Deterministic memetic proxy curves used only to show how fidelity, fecundity, and longevity could be operationalized in later work. Seed = 37.

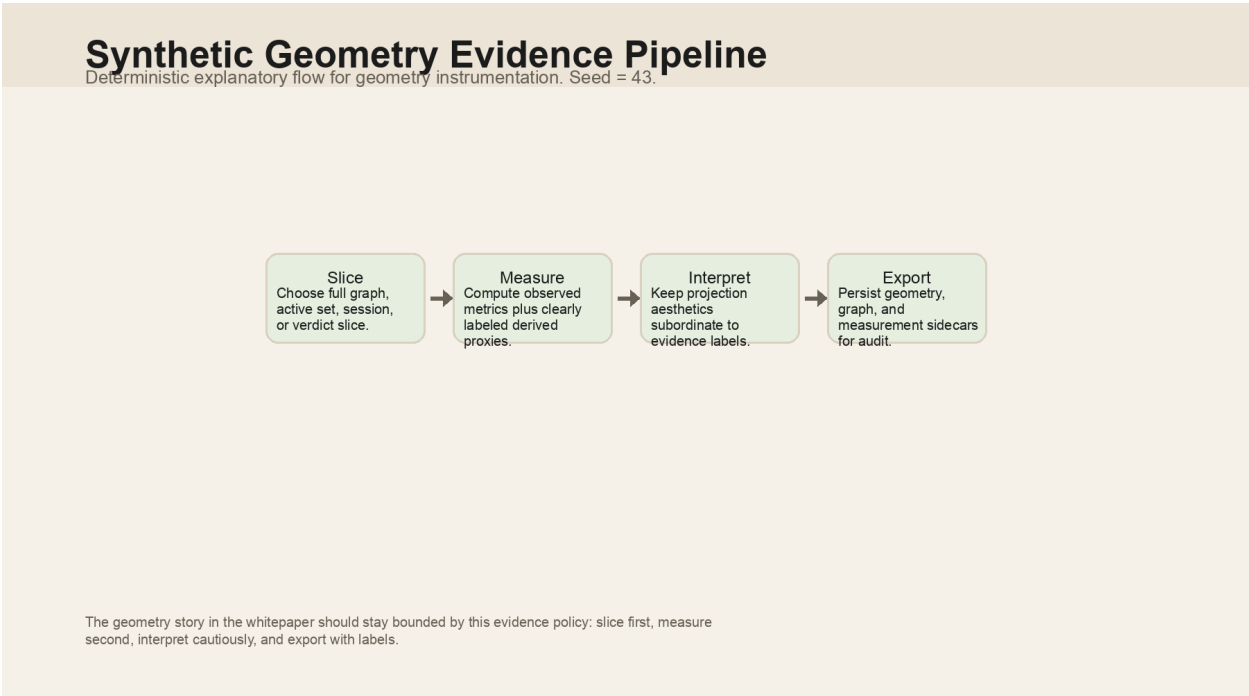

Figure 14: [SYNTHETIC] Deterministic geometry-instrumentation flow used to keep the paper's explanation aligned with the repo's evidence policy. Seed = 43.

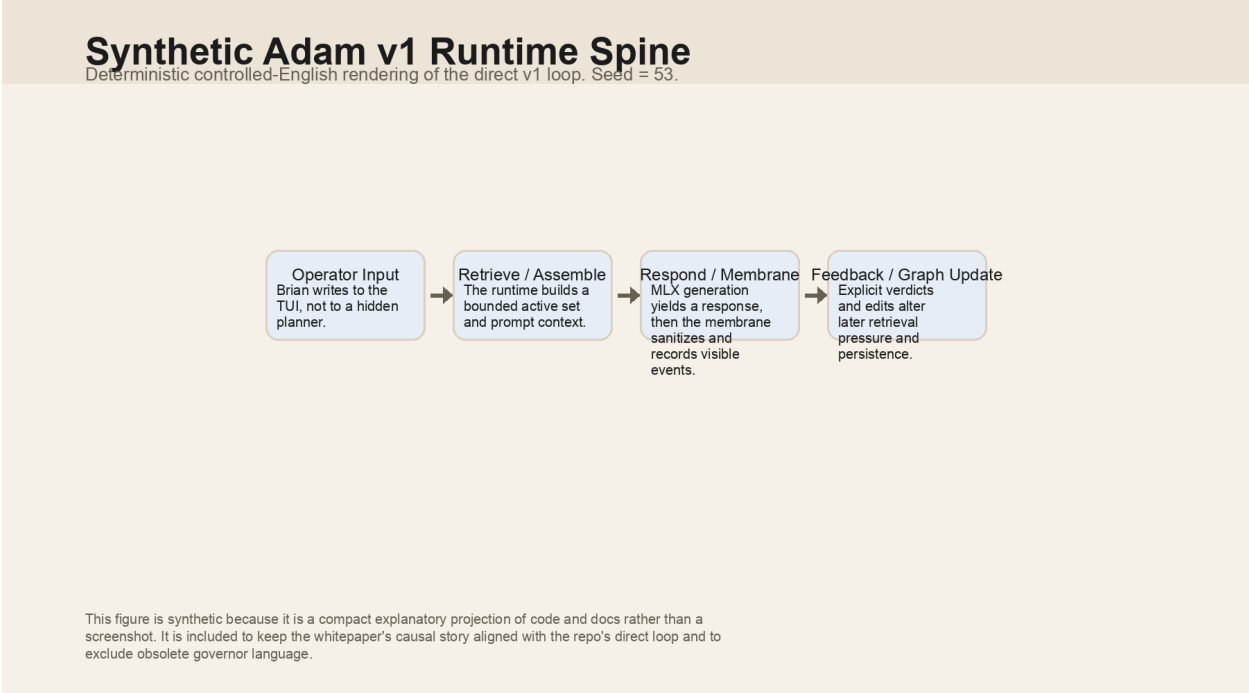

Figure 15: [SYNTHETIC] Deterministic controlled-English rendering of the direct Adam v1 loop. Seed = 53.